

[CS230] Shallow Neural Networks: Activation Functions

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network - *Completed*

Chapter 3. Shallow Neural Networks (Current Unit)

- 3.1 Concept of Hidden Layer & Architecture - *Completed*
- **3.2 Activation Functions (Sigmoid, Tanh, ReLU)**
 - * The Big 4 Functions
 - * Why Sigmoid Failed? (Vanishing Gradient)
 - * Professor's Choice (Best Practice)
 - * Implementation
- 3.3 Backpropagation Intuition

Chapter 4. Deep Neural Networks - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 은닉층(Hidden Layer)을 추가하여 신경망의 깊이를 더했습니다. 그때 제가 "은닉층에는 Sigmoid보다 Tanh나 ReLU가 좋다"고 스쳐 지나가듯 말했습니다. "왜요? Sigmoid가 가장 유명하지 않나요?" 이 질문에 답하지 못하면 여러분은 매번 모델을 설계할 때마다 '선택장애'에 시달릴 것입니다. 활성화 함수는 단순한 스위치가 아닙니다. 학습 신호(Gradient)를 살릴 수도, 죽일 수도 있는 생명 유지 장치입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 딥러닝 모델의 성능을 결정짓는 '**4대 활성화 함수**'를 완벽하게 해부합니다.

- **비교:** Sigmoid, Tanh, ReLU, Leaky ReLU의 수식과 그래프 특징을 비교합니다.
- **원리:** 깊은 신경망에서 Sigmoid를 쓰면 학습이 멈추는 기울기 소실(**Vanishing Gradient**) 문제를 수학적으로 증명합니다.
- **전략:** 출력층과 은닉층에 각각 어떤 함수를 써야 하는지 **Best Practice**를 확립합니다.
- **구현:** NumPy를 사용하여 각 함수와 그 도함수(Derivative)를 효율적으로 코딩합니다.

2 Essential Terminology

함수명	범위	한 줄 특징
Sigmoid	$(0, 1)$	확률 표현에 최적. 하지만 깊어지면 학습 불가.
Tanh	$(-1, 1)$	Sigmoid의 확장판. 0 중심(Zero-centered)이라 학습이 더 빠름.
ReLU	$[0, \infty)$	딥러닝의 표준. 양수는 그대로, 음수는 차단. 연산 빠름.
Leaky ReLU	$(-\infty, \infty)$	ReLU의 변형. 음수일 때도 아주 약간의 기울기를 줌.

3 Core Concepts: The Big 4 Functions

3.1 1. Sigmoid Function (σ)

[Image of sigmoid function graph with equation]

- 수식: $a = \frac{1}{1+e^{-z}}$
- 특징: 0과 1 사이의 값으로 압축합니다. '확률' 개념과 잘 맞습니다.
- 치명적 단점: 입력값(z)이 아주 크거나 작으면 기울기(미분값)가 0에 가까워집니다. 학습이 멈춥니다.
- 용도: 이진 분류의 출력층(**Output Layer**)에만 씁니다. 은닉층엔 절대 쓰지 마세요.

3.2 2. Tanh (Hyperbolic Tangent)

- 수식: $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- 특징: Sigmoid를 위아래로 늘려 -1에서 1 사이 값을 갖게 했습니다. **평균이 0(Zero-centered)** 이므로 데이터의 중심을 잘 잡아주어 Sigmoid보다 학습 수렴이 빠릅니다.
- 용도: 은닉층에서 Sigmoid보다 무조건 좋습니다. 하지만 여전히 기울기 소실 문제는 있습니다.

3.3 3. ReLU (Rectified Linear Unit) - The King

- 수식: $a = \max(0, z)$
- 특징: 단순 무식해 보이지만 가장 강력합니다.
 - $z > 0$: 기울기가 항상 **1**입니다. (신호가 약해지지 않음)
 - $z \leq 0$: 값을 0으로 차단합니다. (불필요한 신호 제거)
- 용도: 모든 은닉층의 기본값(Default)입니다. 고민될 땐 무조건 ReLU를 쓰세요.

□ 전등 스위치 비유 (직관적 비유)

- **Sigmoid**: 조광기(Dimmer). 밝기를 0%에서 100%까지 미세하게 조절하지만, 너무 복잡합니다.
- **ReLU**: 똑딱 스위치. 켜지면 확실하게 켜지고(그대로 통과), 꺼지면 확실하게 꺼집니다(0). 단순함이 속도의 비결입니다.

4 Deep Dive: 왜 Sigmoid는 퇴출당했나?

이 부분은 딥러닝 역사에서 가장 중요한 전환점 중 하나인 기울기 소실 문제(Vanishing Gradient Problem)를 다룹니다.

4.1 The Vanishing Gradient Problem

역전파(Backpropagation)는 출력층의 오차를 입력층까지 전달하기 위해 미분값(기울기)을 계속 곱하는(Chain Rule) 과정입니다.

□ 수학적 증명: 0.25 vs 1.0 (수학적 증명 & 시나리오)

Sigmoid 함수의 미분 최댓값은 $z = 0$ 일 때 **0.25**입니다. 만약 은닉층이 10개라고 가정해봅시다.

Case 1: Sigmoid 사용

$$Gradient \approx 0.25 \times 0.25 \times \dots \times 0.25 = (0.25)^{10} \approx 0.0000009$$

→ 입력층에 도달할 때쯤 기울기는 0이 되어 사라집니다. 앞단은 학습이 전혀 안 됩니다.

Case 2: ReLU 사용 (양수 구간)

$$Gradient = 1 \times 1 \times \dots \times 1 = 1^{10} = 1$$

→ 기울기가 줄어들지 않고 생생하게 입력층까지 전달됩니다. 이것이 100층짜리 딥러닝이 가능한 이유입니다.

5 Professor's Cheat Sheet (Best Practice)

실전에서 무엇을 쓸지 고민하지 마십시오. 이 규칙을 따르면 상위 10%입니다.

활성화 함수 선택 가이드

- 출력층 (Output Layer):
 - 이진 분류 (0 or 1): **Sigmoid**
 - 다중 분류 (Cat, Dog, Bird...): **Softmax**
 - 회귀 (집값 예측): **Linear** (활성화 함수 없음)
- 은닉층 (Hidden Layer):
 - 기본 (Default): **ReLU**
 - ReLU 성능이 아쉽거나 뉴런이 죽는 경우: **Leaky ReLU**
 - 데이터가 매우 적고 모델이 얇을 때: **Tanh**
 - 금지: **Sigmoid** (절대 사용 금지)

6 Implementation (Python with NumPy)

함수값뿐만 아니라 역전파에 필요한 도함수(Derivative)까지 구현합니다.

```
1 import numpy as np
2
3 class Activations:
4     @staticmethod
5     def sigmoid(z):
6         """출력층용: 0 ~ 1"""
7         return 1 / (1 + np.exp(-z))
8
9     @staticmethod
10    def sigmoid_derivative(z):
11        """Sigmoid 미분: a * (1-a)"""
12        s = 1 / (1 + np.exp(-z))
13        return s * (1 - s)
14
15    @staticmethod
16    def relu(z):
17        """은닉층용: max(0, z)"""
18        return np.maximum(0, z)
19
20    @staticmethod
21    def relu_derivative(z):
22        """
23        ReLU 미분:
24        z > 0 이면 1, z <= 0 이면 0
25        """
26        dZ = np.array(z, copy=True) # 원본 보존
27        dZ[z <= 0] = 0
28        dZ[z > 0] = 1
29        return dZ
30
31    @staticmethod
32    def tanh(z):
```

```

33     """은닉층용: -1 ~ 1"""
34     return np.tanh(z) # NumPy 최적화 함수 사용
35
36     @staticmethod
37     def tanh_derivative(z):
38         """Tanh 미분: 1 - a^2"""
39         return 1 - np.power(np.tanh(z), 2)

```

Listing 1: Activation Functions & Derivatives

7 FAQ & Pitfalls

ReLU의 미분 불가능 점 (z)

Q. 수학적으로 $z = 0$ 에서 ReLU는 미분이 불가능한데(뿔죽점), 코딩은 어떻게 하나요?

A. 맞습니다. 하지만 컴퓨터 공학에서는 실용적으로 접근합니다. $z = 0$ 일 때 기울기를 그냥 0이나 1 중 하나로 정해버립니다. (보통 0으로 둡). z 가 정확히 0.0000...이 될 확률은 매우 낮으므로 학습에 아무런 지장이 없습니다.

Q. Leaky ReLU는 언제 쓰나요?

A. ReLU를 썼는데 학습 중에 뉴런의 출력이 계속 0만 나와서 죽어버리는 현상(Dying ReLU)이 발생할 때 씁니다. 음수일 때 0.01 같은 작은 기울기를 주어 뉴런을 소생시킵니다.

다음 단계 (Next Step)

이제 우리는 뉴런의 구조(Layer)와 신호를 조절하는 스위치(Activation)까지 모두 갖췄습니다. 자동차로 치면 엔진과 변속기를 조립한 상태입니다.

다음 시간에는 이 자동차를 실제로 달리게 만드는 엔진 점화 과정, 즉 오차를 줄이기 위해 미분을 사용하는 '역전파(Backpropagation)'의 수식적 유도 과정을 아주 깊이 있게 파헤쳐 보겠습니다. 긴장하십시오. 이제 진짜 미분의 숲으로 들어갑니다.

□ 단원 요약 (Cheat Sheet)

1. **Sigmoid:** 출력층(이진 분류)에만 사용. 은닉층 사용 시 기울기 소실 발생.
2. **ReLU:** 은닉층의 **Default**. 양수는 그대로(기울기 1), 음수는 0. 연산 빠름.
3. **Tanh:** Sigmoid보다 좋음(Zero-centered). 얇은 모델에 적합.
4. **Vanishing Gradient:** Sigmoid 미분값이 1보다 작아(≤ 0.25), 층이 깊어지면 학습 신호가 사라지는 현상.